Arquitetura cliente x servidor: Cliente e Servidor são construídos separadamente, ou seja, um REST nunca será um monólito.

- **Cliente:** É o dispositivo ou programa (como um navegador web, aplicativo de celular ou software de desktop) que **inicia a requisição**, exibe a interface e interage com o usuário. [1, 2]
- **Servidor:** É o sistema centralizado (computador ou máquina na nuvem) que **recebe, processa e atende** as solicitações, gerenciando os dados, a lógica de negócios e os recursos da rede. [1, 2]

Como Funciona o Ciclo

1. O **cliente** envia uma solicitação (requisição) por meio de um protocolo de rede (como HTTP, FTP ou SMTP).
2. O **servidor** recebe essa mensagem, processa os dados (muitas vezes consultando um banco de dados) e executa a lógica necessária.
3. O **servidor** envia uma resposta de volta ao cliente.
4. O **cliente** recebe a resposta e a exibe na tela para o usuário (por exemplo, renderizando uma página da web). [1, 2]

Exemplo prático: Quando você acessa o Gmail no navegador, o seu navegador é o *cliente* e os computadores do Google na nuvem atuam como *servidores* processando e enviando os seus e-mails. [1]

Vantagens

- **Centralização de dados:** Facilita backups, controle de versão e segurança das informações em um único lugar.
- **Escalabilidade:** É possível aumentar a capacidade do servidor ou adicionar novos servidores conforme o tráfego cresce.
- **Manutenção simplificada:** Atualizações e correções de bugs na lógica principal são feitas diretamente no servidor, sem precisar alterar cada máquina cliente. [1, 2, 3]

Desvantagens

- **Dependência do servidor:** Se o servidor cair ou sofrer uma sobrecarga, todos os clientes que dependem dele ficam sem acesso ao serviço.
- **Gargalos e latência:** Um grande volume de requisições simultâneas pode sobrecarregar o servidor e tornar a resposta mais lenta.

- **Custo de infraestrutura:** Exige servidores robustos e manutenção contínua de rede. [1, 2, 3]

Para se aprofundar nos fundamentos teóricos, você pode consultar o artigo da DIO ou a página da Wikipédia sobre o modelo cliente-servidor. [1, 2]

Comunicação Stateless: Como vimos no conteúdo de estado, uma comunicação Stateless significa que cada interação entre cliente e servidor é feito de forma independente, ou seja, o sistema nunca lembra (salva) alguma interação. vamos aprofundar um pouco mais nisso:

As principais características desse modelo de comunicação incluem:

- **Independência absoluta:** O servidor não armazena nenhum histórico ou contexto de sessão do usuário. Para o sistema, cada requisição chega "do zero", como se fosse a primeira vez. [1, 2, 3, 4]
- **Autossuficiência de dados:** Como o servidor "esquece" tudo logo após responder, a requisição enviada pelo cliente deve obrigatoriamente conter todas as informações necessárias para ser processada (ex: credenciais, tokens de autenticação, IDs e parâmetros). [1, 2]
- **Facilidade de escalabilidade horizontal:** Esta é uma das maiores vantagens. Como nenhum servidor guarda dados da sessão na memória local, as requisições podem ser distribuídas livremente por um balanceador de carga para qualquer máquina disponível no cluster, sem quebrar o fluxo do usuário. [1, 2]
- **Simplificação do servidor:** O design da infraestrutura se torna mais leve e fácil de gerenciar. O servidor não precisa alocar memória para gerenciar o tempo de expiração de sessões ativas ou sincronizar esses estados entre múltiplos servidores. [1]
- **Resiliência a falhas:** Se um servidor cair no meio do caminho, o cliente pode simplesmente reenviar a requisição para outro servidor ativo, pois nenhuma informação contextual crucial foi perdida com a queda da máquina anterior. [1, 2]

Exemplos práticos:

Os protocolos e padrões que sustentam a internet moderna funcionam sob essa filosofia. O protocolo **HTTP** (utilizado para navegar na web), o protocolo **IP**, requisições de DNS, além do estilo arquitetural de **APIs REST**, funcionam nativamente de forma stateless. Quando os sites precisam lembrar de você (como um carrinho de compras),

eles usam mecanismos adicionais no lado do cliente, como *cookies* ou tokens JWT. [1, 2, 3, 4]

Cache: O cache em uma **API REST** é a capacidade de armazenar cópias de respostas de dados acessados com frequência em um local de armazenamento temporário (no cliente, em servidores intermediários ou no próprio servidor). [1, 2, 3]

Quando uma nova requisição é feita, o sistema verifica se já existe uma cópia válida e atualizada daquele recurso. Se existir, ela é devolvida de imediato, sem a necessidade de reprocessar a lógica ou consultar o banco de dados novamente. [1, 2, 3]

Principais benefícios

- **Redução de latência:** As respostas chegam muito mais rápido para o usuário final.
- **Menor carga no servidor:** Evita consultas repetitivas e pesadas ao banco de dados ou a serviços externos.
- **Escalabilidade:** Permite que a API suporte muito mais acessos simultâneos usando menos recursos.
- **Constraint REST:** O cache é inclusive uma das diretrizes fundamentais (constraints) da arquitetura REST original. [1, 2, 3, 4]

Como funciona no HTTP (Headers de Cache)

Como o REST é construído sobre o protocolo **HTTP**, o controle de cache é feito principalmente através de cabeçalhos (*headers*) trocados entre o cliente e o servidor: [1, 2]

- `Cache-Control`: Define as regras de armazenamento (ex: `public`, `private`, `no-cache`, ou `max-age=3600` para dizer que o recurso pode ser guardado por 1 hora).
- `ETag` **(Entity Tag):** Um identificador único (uma "assinatura") para a versão de um recurso. Se o recurso não mudou, o servidor responde com o status `304 Not Modified` em vez de reenviar todo o conteúdo.
- `Last-Modified`: Indica a data da última alteração do recurso. [1]

Onde o cache pode ser aplicado?

1. **No Cliente (Navegador ou App):** O próprio dispositivo guarda imagens, JSONs ou páginas para não baixá-los de novo a cada clique. [1, 2]
2. **Em Intermediários (Proxies, CDNs, API Gateways):** Servidores no meio do caminho que atendem requisições de vários usuários na mesma região.

3. **No Servidor (Backend):** Utilizando memórias de alta velocidade ou bancos de dados chave-valor (como o **Redis** ou **Memcached**) para guardar o resultado de consultas caras antes de bater no banco principal. [1, 2, 3]

Você pode saber mais sobre as boas práticas de design e arquitetura no artigo da Speakeasy sobre Cache em APIs REST ou consultar a visão geral da AWS sobre API RESTful. [1, 2]

Interface Uniforme: Uma **interface uniforme** é a restrição mais importante e fundamental da arquitetura **REST** (*Representational State Transfer*), definida por Roy Fielding. Ela padroniza a forma como o cliente e o servidor conversam, garantindo que qualquer componente consiga interagir com outro de maneira previsível, sem acoplamento rígido. [1, 2, 3, 4]

Na prática, baseada em HTTP, essa interface se divide em **quatro pilares principais**: [1, 2]

- **Identificação de recursos por URIs:** Cada recurso (como um usuário, um pedido ou um produto) é único e identificado por um endereço próprio (por exemplo, `/clientes/123`). [1, 2, 3]
- **Manipulação de recursos através de representações:** O cliente não mexe diretamente no banco de dados do servidor; ele recebe e envia *representações* desse recurso (geralmente em formato **JSON** ou **XML**). [1, 2]
- **Mensagens autodescritivas:** Cada requisição e resposta contém todas as informações necessárias para ser compreendida sozinha — incluindo os **métodos HTTP padrão** (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`) e os códigos de status (`200 OK`, `404 Not Found`, etc.). [1, 2, 3]
- **Hipermídia como motor do estado da aplicação (HATEOAS):** Através de links retornados nas respostas, o cliente consegue descobrir dinamicamente quais outras ações ele pode realizar a partir daquele ponto, sem precisar "adivinhar" rotas. [1, 2]

Para mais detalhes sobre boas práticas de modelagem, você pode consultar o Guia de Design de API Web da Microsoft ou a documentação da AWS sobre APIs RESTful. [1, 2]

Sistema de camadas: A restrição de sistema em camadas na arquitetura REST determina que o sistema deve ser composto por camadas hierárquicas, cada uma com sua função específica. Esse design promove a separação de responsabilidades, aumenta a escalabilidade e permite que componentes intermediários melhorem o desempenho e gerenciem a segurança.

Princípios Fundamentais de um Sistema em Camadas

1. **Separação de responsabilidades** : Cada camada lida com uma responsabilidade específica, como armazenamento de dados, lógica de negócios ou apresentação.
2. **Escalabilidade** : Camadas podem ser adicionadas ou removidas conforme necessário para dimensionar o sistema de forma eficiente.
3. **Componentes intermediários** : Componentes intermediários como gateways, firewalls e proxies podem ser inseridos para lidar com tarefas específicas sem afetar outras camadas.

Benefícios de um sistema em camadas

1. **Escalabilidade** : Ao distribuir as tarefas por diferentes camadas, o sistema pode ser dimensionado de forma mais eficaz.
2. **Flexibilidade** : Camadas individuais podem ser modificadas ou substituídas sem afetar todo o sistema.
3. **Segurança** : Camadas intermediárias podem gerenciar aspectos de segurança como autenticação e autorização.
4. **Manutenibilidade** : A clara separação de responsabilidades torna o sistema mais fácil de manter e evoluir.

Camadas em um sistema RESTful

Um sistema RESTful típico pode ter várias camadas, incluindo:

1. **Camada do Cliente** : A interface do usuário ou aplicativo cliente que faz solicitações ao servidor.
2. **Camada de apresentação** : gerencia a lógica de apresentação e a interface do usuário.
3. **Camada de Lógica de Negócios** : Gerencia a lógica e as regras principais da aplicação.
4. **Camada de Acesso a Dados** : Gerencia o armazenamento e a recuperação de dados.
5. **Camada de segurança** : gerencia autenticação, autorização e criptografia.
6. **Camada de Infraestrutura** : Gerencia componentes de rede, balanceamento de carga e outros aspectos da infraestrutura.

Exemplo de um sistema em camadas

Vamos considerar uma aplicação típica de comércio eletrônico. Cada camada tem uma função específica:

1. **Camada do cliente** : O aplicativo web ou móvel onde os usuários navegam pelos produtos e fazem compras.
2. **Camada de apresentação** : O servidor que gera HTML, CSS e JavaScript para o aplicativo web.
3. **Camada de Lógica de Negócios** : O servidor de aplicativos que processa pedidos, gerencia o estoque e administra as contas de usuário.
4. **Camada de Acesso a Dados** : O servidor de banco de dados onde são armazenadas as informações do produto, os dados do usuário e o histórico de pedidos.
5. **Camada de segurança** : Um gateway ou gateway de API que lida com autenticação e autorização.
6. **Camada de Infraestrutura** : Balanceadores de carga, firewalls e componentes de rede que garantem uma operação estável e segura.

Exemplo prático

Eis como um sistema em camadas poderia funcionar na prática para uma aplicação de comércio eletrônico:

1. **Solicitação do cliente** : O usuário navega pelo catálogo de produtos em um aplicativo móvel.
2. **Gateway de API** : A solicitação é inicialmente processada por um gateway de API que autentica o usuário e encaminha a solicitação para o serviço apropriado.
3. **Camada de Lógica de Negócios** : O servidor de aplicativos processa a solicitação, aplicando regras de negócios como a verificação de estoque.
4. **Camada de Acesso a Dados** : O servidor consulta o banco de dados para obter informações sobre o produto e o status do estoque.
5. **Resposta** : O servidor de aplicativos constrói a resposta e a envia de volta ao cliente através do gateway da API.
6. **Camada do cliente** : O aplicativo móvel exibe as informações do produto para o usuário.

Exemplo de interação de sistema em camadas

Solicitação do cliente :

```
GET /products/ 123 HTTP/ 1.1
Host: api.example.com
Authorization: Bearer <token>
```

Gateway de API :

- Valida o token.
- Encaminha a solicitação para o servidor de lógica de negócios.

Servidor de Lógica de Negócios :

- Verifica as regras de negócio e o inventário.
- Obtém os dados do produto a partir da base de dados.

Servidor de banco de dados :

- Recupera informações do produto e as envia de volta para o servidor de lógica de negócios.

Servidor de Lógica de Negócios :

- Elabora a resposta.

Gateway de API :

- Envia a resposta de volta ao cliente.

Resposta do cliente :

```
{
  "id" :  123 ,
  "name" :  "Mouse sem fio" ,
  "price" :  25.99 ,
  "inventory" :  "Em estoque"
}
```

Intermediários em um Sistema em Camadas

1. **Proxies** : Encaminham solicitações do cliente para o servidor, frequentemente usados para balanceamento de carga e armazenamento em cache.
2. **Gateways** : Gerenciam a segurança e encaminham as solicitações para os serviços apropriados.
3. **Firewalls** : Protegem o sistema filtrando o tráfego de entrada e saída.

Exemplo com um proxy :

```
Cliente - > Proxy - > Gateway de API - > Servidor de Lógica de Negócios - >
Servidor de Banco de Dados
```

Conclusão

A restrição de sistema em camadas na arquitetura RESTful promove escalabilidade, flexibilidade, segurança e facilidade de manutenção, organizando o sistema em camadas hierárquicas com responsabilidades específicas. Ao utilizar intermediários e separar as responsabilidades, um sistema em camadas pode gerenciar a complexidade de forma eficiente e lidar com uma ampla gama de tarefas, desde autenticação até processamento e armazenamento de dados.